Phase I
(Setup)

External libraries:-
python-can (import can)
pyisotp (import isotp)
udsoncan (import udsoncan)
openpyxl (import openpyxl)

In-build libraries:-
time (import time)
threading (import threading)

Tool code

Define:
[load_requests_from_excel()
[decode_did_value(did, data)]

↓

Call: [load_requests_from_excel()]

↓

Call: [can.Bus()]
Config: interface='udp_multicast'
Params: channel='-', port=-

↓

Call: [isotp_address()]
Setup: rxid=0x7E8, txid=0x7E0

↓

Call: [isotp.CanStack()]
Argument: bus=bus, address=addr
Params: stmin=0, blocksize=8,
tx_padding=0x00
(standard)

↓

Setup Application Timeout Options
Call: [default_client_config.copy()]
Config: custom_config['p2_timeout'] = 60.0
Config: custom_config['p2_star_timeout'] = 60.0

ECU code

Define:
[load_ev_state()]
[state_polling_thread()]
[process_uds_request(payload)]

↓

Call: [threading.Thread()]
Argument: target=[state_polling_thread]
Argument: daemon=True

↓

Call: [load_ev_state()]
Params: time.sleep(2.0)
(refresh period)

↓

Call: [can.Bus()]
Config: interface='udp_multicast'
Params: channel='-', port=-

↓

Call: [isotp_address()]
Setup: rxid=0x7E0, txid=0x7E8

↓

Call: [isotp.CanStack()]
Argument: bus=bus, address=addr
Params: stmin=0, blocksize=8,
tx_padding=0x00
(standard)

PHASE II
(User request generation)
(Tool code)

Event Listener Active (application in idle state)
call : .get()/.text() & app.exec_()
eg. output : 22 01 05

↓

Build Targeted UDS Request Frame Array
Call: .replace(" ", "")
Call:bytes.fromhex(sanitized_input)]
eg output : b'\x22\x01\x05'

↓

Execution Call: [client.send_request()]

Internal Network Sub-Call: [stack.send()]

↓

TX PIPELINE: ISO-TP
Slices Multi-frame
Data Packets

Phase III
(The ECU Interpretation)
(ECU code)

(ECU Main Loop) Running Pipeline:
Call: [stack.process()]
Params: time.sleep(0.005) (Protects system CPU load)

⬇

Check & Receive Network Payload
Condition: if [stack.available()]: (Returns True)
Call: payload = [stack.recv()]

⬇

Call: [process_uds_request()]
Argument: payload
Variable Extraction: sid = payload[0]
Interaction Call: choice = [input()]

if Negative Response

⬇

Manual Error Injection
Action: Builds 3-byte error array
Return Call: bytes([0x7F, sid, 0x22])

if Positive Response

⬇

SID 0x10
Variable: subfunc = payload[1]
Return Call:
[bytes([sid + 0x40, subfunc])]

SID 0x22
Bitwise Parse: did = (p[1]<<8)\|p[2]
if did in EV_STATE_MEMORY, fetches data array from RAM
Return Call:
[bytes([0x62, p1, p2]) + data]

SID 0x19
Read active fault array matrix and packs the reporting type byte
Return Call:
[bytes([sid + 0x40, subfunc]) + dtc_hex_block]

⬇

Internal Return Endpoint:
Assignment: response_payload = (Compiled branch bytes array output container)

⬇

Call: [stack.send(response_payload)]
Config: Encapsulates payload array container into an ISO-TP network frame segment.

PHASE IV
(Decoding and Showing the result)
(Tool code)

Method Call: [client.send_request()]
Returns
eg input : b'\x62\x01\x05\x55'

(If response data byte starts with 0x7F)

(If response indicates a positive completion flag)

Conditional Safety Exception Match Blocks:
Call: [print()] "Request rejected by ECU!..."
OR
Call: [print()] "ECU failed to respond within P2" OR
Call: [print()] "Unexpected error: {e}"
eg input : 0x7F ;
output : “Request rejected by ECU!”

Slicing Data Truncation
Operation:actual_data=response.data[2:]
eg Input: b'\x62\x01\x05\x55' ;
output :b'\x55'

Metrics Translation Call: [decode_did_value()]
Call: [int.from_bytes(..., 'big')]
input: b’\55 ; output: 85

(Tool GUI Context) End-User Presenter Node
call : my_textedit.append(final_string)
eg: “SOC is 85%”

Software architecture of open source EV diagnostic engine

External libraries:-
python-can (import can)
pyisotp (import isotp)
udsoncan (import udsoncan)
openpyxl (import openpyxl)

In-build libraries:-
time (import time)
threading (import threading)

Define:
[load_requests_from_excel()
[decode_did_value(did, data)]

Call: [load_requests_from_excel()]

Call: [can.Bus()]
Config: interface='udp_multicast'
Params: channel='-', port=-

Call: [isotp_address()]
Setup: rxid=0x7E8, txid=0x7E0

Call: [isotp.CanStack()]
Argument: bus=bus, address=addr
Params: stmin=0, blocksize=8, tx_padding=0x00
(standard)

Setup Application Timeout Options
Call: [default_client_config.copy()]
Config: custom_config['p2_timeout'] = 60.0
Config: custom_config['p2_star_timeout'] = 60.0

Define:
[load_ev_state()]
[state_polling_thread()]
[process_uds_request(payload)]

Call: [threading.Thread()]
Argument: target=[state_polling_thread]
Argument: daemon=True

Call: [load_ev_state()]
Params: time.sleep(2.0)
(refresh period)

Call: [can.Bus()]
Config: interface='udp_multicast'
Params: channel='-', port=-

Call: [isotp_address()]
Setup: rxid=0x7E0, txid=0x7E8

Call: [isotp.CanStack()]
Argument: bus=bus, address=addr
Params: stmin=0, blocksize=8, tx_padding=0x00
(standard)

(ECU Main Loop) Running Pipeline:
Call: [stack.process()]
Params: time.sleep(0.005) (Protects system CPU load)

Check & Receive Network Payload
Condition: if [stack.available()]: (Returns True)
Call: payload = [stack.recv()]

Call: [process_uds_request()]
Argument: payload
Variable Extraction: sid = payload[0]
Interaction Call: choice = [input()]

if Negative Response

if Positive Response

Manual Error Injection
Action: Builds 3-byte error array
Return Call: bytes([0x7F, sid, 0x22])

SID 0x10
Variable: subfunc = payload[1]
Return Call:
[bytes([sid + 0x40, subfunc])]

SID 0x22
Bitwise Parse: did = (p[1]<<8)\|p[2]
if did in EV_STATE_MEMORY, fetches data array from RAM
Return Call:
[bytes([0x62, p1, p2]) + data]

SID 0x19
Read active fault array matrix and packs the reporting type byte
Return Call:
[bytes([sid + 0x40, subfunc]) + dtc_hex_block]

Internal Return Endpoint:
Assignment: response_payload = (Compiled branch bytes array output container)

Call: [stack.send(response_payload)]
Config: Encapsulates payload array container into an ISO-TP network frame segment.

TX PIPELINE: ISO-TP
Slices Multi-frame
Data Packets

Event Listener Active (application in idle state)
call : .get()/.text() & app.exec_()

Build Targeted UDS Request Frame Array
Call:
bytes.fromhex(sanitized_input)]

Execution Call: [client.send_request()]

Internal Network Sub-Call: [stack.send()]

TX PIPELINE: ISO-TP Slices Multi-frame
Data Packets

Method Call: [client.send_request()]
Returns
eg input : b'\x62\x01\x05\x55'

(If response data byte starts with 0x7F)

(If response indicates a positive completion flag)

Conditional Safety Exception Match Blocks:
Call: [print()] "Request rejected by ECU!..." OR
Call: [print()] "ECU failed to respond within P2"
OR
Call: [print()] "Unexpected error: {e}"
eg input : 0x7F ;
output : “Request rejected by ECU!”

Slicing Data Truncation
Operation:actual_data=response.data[2:]
eg Input: b'\x62\x01\x05\x55' ;
output :b'\x55'

Metrics Translation Call: [decode_did_value()]
Call: [int.from_bytes(..., 'big')]
input: b’\55 ; output: 85

(Tool GUI Context) End-User Presenter Node
call : my_textedit.append(final_string)
eg: “SOC is 85%”